Project Statement: YouTube Video Intelligence MCP Server

The Problem: The Noise of the Algorithm

When users seek to master complex concepts, YouTube's engagement algorithm often pushes distracting content (e.g., viral videos, unrelated streams) instead of structurally similar, high-quality educational material. Compiling a focused playlist requires opening multiple tabs, manually checking runtimes, evaluating creators, and compiling lists. Users need an intelligent assistant that understands context, respects time limits, and cuts out algorithmic noise.

The Solution: AI-Driven Curation via MCP

This project provides a conversational AI with the ability to securely act on the user's behalf using the Model Context Protocol (MCP). The user asks the AI to build a playlist based on strict constraints (e.g., 'Build a two-hour playlist on this topic, with no video over 20 minutes'). The MCP server connects the AI to YouTube's official Data API v3, allowing the AI to filter videos based strictly on objective metadata rather than engagement algorithms, and automatically creates the tailored playlist directly in the user's account.

1. Project Description

Build an MCP server that lets an AI client discover, analyse, and curate YouTube videos for an authorised user. The system shall integrate with the YouTube Data API v3. It shall expose tools, resources, and Agent Skills through an MCP-compatible server. An AI client (for example Claude, ChatGPT, or Gemini) shall use them to retrieve YouTube information, perform basic analysis, and carry out selected YouTube operations.

2. Functional Requirements

- **FR-1 (Similar Videos):** Given a video URL or ID, return a ranked list of similar videos.
- **FR-2 (Similar Channels):** Given a channel URL, handle, or ID, return a ranked list of similar channels.
- **FR-3 (Recommendations):** Recommend videos based on the authorised user's liked videos, subscriptions, and playlists.
- **FR-4 (Engagement Analysis):** Analyse the user's liked videos, subscriptions, and playlists: top channels and categories, trends over time, duration preferences, and comparisons between periods.
- **FR-5 (Playlist Creation):** Accept a natural-language request with constraints (topic, total duration, maximum video length, language). Build a playlist that satisfies them,

show it to the user, and create it in the user's YouTube account only after explicit confirmation.

- **Implementation Note:** The YouTube Data API does not give watch history and has no related-videos or related-channels endpoint. Teams shall compute similarity themselves from video and channel metadata using simple techniques (TF-IDF, cosine similarity over feature vectors, basic weighted scoring).

3. Product-Specific Agent Skills

- **video-to-playlist:** Run a complete product workflow from one video: 1. Summarise a given video. 2. Find similar videos (FR-1). 3. Propose a playlist and ask the user to confirm. 4. Create the playlist (FR-5). 5. Show a short analysis of the playlist.

4. Product-Specific Testing Requirements

- **TQ-1:** Unit tests for functions, modules, services, and MCP tools. Include the similarity scoring, duration parsing, and playlist constraint logic.
- **TQ-2:** Integration tests for interactions between components, including the YouTube provider.
- **TQ-3:** System (end-to-end) tests for each of FR-1 to FR-5, driven through the MCP interface.
- **TQ-4:** Negative tests: invalid/malformed IDs, private/deleted videos, missing tags, quotaExceeded, youtubeSignupRequired, expired tokens, and rate limiting.
- **TQ-5:** CI tests shall run against recorded fixtures or mocks, not the live API. Run live API tests in a separate scheduled workflow with a fixed quota budget.
- **TQ-6:** The system shall track quota cost per request. Test that a workflow which would exceed the remaining budget is rejected before any API call.
- **Security Testing:** OAuth (minimum scopes, safe storage), Secrets (none in repo), Input validation (all parameters), Prompt injection (ensure untrusted video metadata cannot bypass user confirmation), Personal data (no real user data in fixtures/logs).

5. API Constraints and Compliance

- **Official API Only:** Use only the official YouTube Data API v3. Scraping or unofficial APIs are not permitted.
- **Quota Limits:** A new project has a default of 10,000 units per day. Use caching and fixtures. Do not create extra Google Cloud projects to get more quota.
- **OAuth in Testing:** Refresh tokens in Testing mode expire after 7 days; design for re-authorisation.
- **Channel Required:** A Google account without a YouTube channel cannot create playlists. Handle this case (TQ-4).